DEPI Round 4

Final Project

Medora

Table of content

Team Members & Roles

Name
Salma Fahmy Hassan
Rahma Ramadan Hassan
Mohamed Atef
Marihan Emad
Ahmed Samy Eissa
Ali Soliman

Instructor: Aya Hisham **Track:** Generative AI

1. Introduction and Background

1.1 Introduction

Artificial Intelligence has become one of the most influential technologies in the healthcare sector, enabling faster and more accurate medical assistance. Medora is an AI-powered healthcare platform designed to help users understand their medical conditions through symptom analysis, medical document processing, and intelligent healthcare recommendations.

The system combines multiple AI technologies, including Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), Optical Character Recognition (OCR), and Computer Vision, to analyze medical reports, prescriptions, laboratory results, and medical images. In addition, Medora provides drug interaction checking, nutrition and rehabilitation recommendations, and doctor suggestions based on the user's condition, making healthcare information more accessible and easier to understand.

1.2 Problem Definition

1.2.1 Overall Problem Statement

Many people struggle to obtain reliable medical information and often rely on inaccurate online sources. Understanding medical reports, prescriptions, and symptoms without professional assistance can be difficult, which frequently leads to confusion and delayed healthcare decisions.

1.2.2 Challenges in Accessing Reliable Medical Guidance

- Difficulty understanding medical reports and prescriptions.
- Widespread inaccurate medical information online.
- Limited access to quick medical consultation.
- Lack of integrated platforms that combine symptom analysis, document interpretation, and healthcare recommendations.

1.2.3 Importance of Addressing the Problem

Providing an intelligent healthcare assistant can improve patient awareness, simplify medical information, support early health assessment, and help users make more informed decisions before consulting healthcare professionals.

1.3 Motivation

Improving Healthcare Accessibility

Provide users with instant access to reliable medical guidance anytime and anywhere.

Assisting Early Symptom Assessment

Help users better understand possible health conditions based on their symptoms and uploaded medical documents.

Supporting Medical Decision-Making

Assist users by explaining medical reports, identifying drug interactions, and providing evidence-based recommendations.

Enhancing Patient Awareness

Simplify complex medical information into clear and understandable explanations.

Integrating AI with Medical Documents and Images

Use OCR, Computer Vision, and AI models to analyze medical reports, prescriptions, and medical images.

Providing Personalized Health Recommendations

Generate personalized nutrition advice, rehabilitation suggestions, medication information, and doctor recommendations according to the user's condition.

1.4 Aims of Work

The main aim of Medora is to develop an AI-powered healthcare platform that provides users with reliable preliminary medical assistance through intelligent symptom analysis, medical document processing, and personalized healthcare recommendations. The system aims to improve healthcare accessibility while supporting informed medical decision-making.

1.5 Objectives

The objectives of Medora are to:

- Develop an intelligent AI medical chatbot for healthcare assistance.
- Analyze medical reports, prescriptions, and laboratory results.
- Process medical images using AI-based image analysis.
- Retrieve reliable medical information using RAG technology.
- Detect potential drug interactions.
- Provide personalized nutrition and rehabilitation recommendations.
- Recommend suitable doctors based on the user's condition and location.
- Ensure safe and reliable AI responses through validation mechanisms.

1.6 Project Plan

Overview

The development of Medora followed a structured process that included research, system design, implementation, testing, and evaluation to ensure an efficient and reliable healthcare platform.

Research and Requirement Analysis

Studying AI technologies, healthcare systems, and user requirements, and selecting suitable tools and models for the project.

System Design

Designing the system architecture, database structure, AI workflow, and user interface.

AI Model Integration

Integrating Large Language Models (LLMs), RAG, OCR, and Computer Vision into a unified multimodal healthcare system.

Development and Implementation

Developing the backend APIs, frontend interface, AI modules, and healthcare recommendation services.

Testing and Evaluation

Testing all system components, validating AI responses, and evaluating overall system performance and accuracy.

1.7 Project Scope

The scope of Medora includes:

- AI-powered medical chatbot.
- Medical document analysis.
- Laboratory report interpretation.
- Prescription processing.
- Medical image analysis.
- Symptom assessment using AI.

- Retrieval-Augmented Generation (RAG) for medical knowledge.
- Drug interaction checking.
- Personalized nutrition recommendations.
- Rehabilitation and exercise recommendations.
- Egyptian doctor recommendation system.
- Safety validation and response verification.
- System testing and performance evaluation.

2. Literature Survey

2.1 Related Work & Previous Solutions

2.1.1 Traditional Healthcare Consultation

Traditional healthcare consultation relies on in-person visits to clinics or hospitals, where a physician manually reviews the patient's symptoms, history, and any laboratory or imaging results before reaching a diagnosis. While this model benefits from direct human expertise and professional judgment, it suffers from several practical limitations: appointment waiting times can be long, consultation slots are limited, and access is heavily dependent on geographic proximity to qualified specialists. Patients frequently need to visit several different specialists (e.g., a nutritionist, a physiotherapist, and a pharmacist) to obtain the full picture of care around a single condition, which increases both cost and time before a coherent treatment plan is reached. In addition, patients are often given verbal or handwritten instructions that are not systematically stored, making it harder to track medication history, past diagnoses, or lab trends over time. These limitations motivate the need for a digital, always-available layer that can provide a preliminary understanding of a condition and consolidate related recommendations before or alongside a real consultation.

2.1.2 Existing AI Healthcare Assistants

Many existing AI healthcare assistants primarily focus on symptom assessment through conversational interfaces, typically allowing users to describe their symptoms in natural language and receive a list of possible conditions along with generic advice on next steps. However, most of these systems provide limited support for multimodal medical document understanding, medical image analysis, personalized healthcare recommendations, and integrated clinical workflows. They generally do not integrate document understanding (e.g., reading an uploaded lab report or prescription), do not check for drug interactions, and do not extend into related domains such as nutrition or rehabilitation. Furthermore, many of these assistants operate as closed, static knowledge systems without a retrieval-augmented generation (RAG) layer, which increases the risk of outdated or unsupported answers, and few of them provide any explicit safety validation or disclaimer layer around the generated response.

2.1.3 Medical Document Analysis Systems

Medical document analysis systems typically rely on Optical Character Recognition (OCR) to convert scanned prescriptions, lab reports, or discharge summaries into machine-readable text, which is then parsed using rule-based patterns or, more recently, large language models to extract structured fields such as patient information, medications, and lab values. Standalone OCR pipelines (e.g., engines such as PaddleOCR or EasyOCR) are effective at raw text extraction but are not inherently capable of clinical reasoning, meaning that a separate parsing and interpretation layer is required to turn extracted text into clinically meaningful entities such as diagnoses, medications, and lab flags. Systems that combine OCR with a downstream medical-entity parser and a reference-range based lab interpretation engine are able to close this gap and produce a structured, queryable representation of the uploaded document rather than plain extracted text.

2.1.4 AI-Based Medical Image Analysis Systems

Beyond document OCR, a separate category of systems focuses on analyzing medical images directly, most notably for dermatological and wound-care use cases. These systems use vision-capable models — either general-purpose multimodal LLMs or specialized image-captioning models — to describe visual characteristics of a lesion or wound (color, texture, shape, size, distribution) and to propose a ranked list of likely conditions rather than a single rigid label. Because dedicated medical imaging models can be costly or unavailable, many practical systems adopt a tiered approach: a stronger vision-language model is used as the primary analyzer, with a lightweight, freely available image-captioning model kept as a fallback to guarantee availability even when the primary provider's API key or quota is unavailable. This tiered fallback design improves reliability and is a pattern that recurs across most of the pipeline components used in modern multimodal medical assistants.

2.2 Our Solution

2.2.1 Medora Overview

Medora is an AI-powered healthcare platform that helps users understand their medical conditions through symptom analysis, medical document processing, and intelligent healthcare recommendations. Unlike single-purpose symptom checkers, Medora combines Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), Optical Character Recognition (OCR), and Computer Vision into one unified pipeline so that a single conversation can be grounded either in the user's own words or in a document/image they upload (a lab report, prescription, or photo of a skin or wound condition). The extracted clinical context — diagnoses, symptoms, medications, and lab values — flows automatically into every downstream recommendation service, ensuring that the chatbot's answer, the drug-interaction check, the nutrition plan, the rehabilitation suggestions, and the doctor recommendations are all grounded in the same underlying clinical picture.

2.2.2 Key Features of Medora

Medora's solution is organized around six major pillars:

- AI Medical Chatbot (RAG-based) — a conversational layer that answers free-text medical questions using retrieval-augmented generation over a medical knowledge base.
- Multimodal Document Understanding — vision- and OCR-based extraction of structured clinical data from uploaded lab reports, prescriptions, and general medical documents.
- Medical Image Analysis — a dedicated analyzer for skin and wound conditions that returns a likely medical condition, ranked alternative possibilities, visual observations, and care guidance.
- Drug Interaction Checking — a rules-based interaction checker together with an LLM-backed drug information branch.
- Personalized Nutrition & Rehabilitation Recommendations — condition- and symptom-aware lifestyle, diet, and exercise guidance generated in parallel with the main chat response.
- Egyptian Doctor Recommendation — a geo-aware doctor search that matches recommended specialties to nearby, highly rated doctors.

All six pillars share the same clinical context object, so a diagnosis extracted from an uploaded document is automatically reused as the grounding symptom set for the drug, nutrition, rehab, and doctor-recommendation branches, without requiring the user to re-describe their condition to each sub-service.

2.2.3 Comparison with Existing Solutions

The table below summarizes how Medora differs from traditional consultation and from typical single-purpose AI health assistants.

Capability	**Traditional / Generic AI Assistants**	**Medora**
Symptom analysis	Basic questionnaire or single LLM	RAG-grounded chatbot + vision/OCR symptom extraction
Document understanding	Rarely supported	Dedicated vision + OCR + parsing pipeline
Medical image analysis	Not supported	Dedicated wound/skin image analyzer with fallback model
Drug interaction checking	Not supported	Rules-based interaction checker
Nutrition & rehab guidance	Not supported / generic	Condition-specific, generated in parallel with chat
Doctor recommendation	Not supported	Geo-aware Egyptian doctor search
Response safety validation	Rarely explicit	Dedicated validation, guardrails, and disclaimer layer

2.3 Stakeholders

The following stakeholder groups interact with, or are impacted by, the Medora platform:

- Patients — the primary end users, who upload documents/images or describe symptoms and receive explanations, lifestyle recommendations, and doctor suggestions.
- Doctors — benefit indirectly through better-informed patients, and are the target of the doctor-recommendation feature, which surfaces suitable specialists to patients based on condition and location.
- Healthcare Providers — clinics and independent practitioners who may be listed in, or referred to by, the doctor recommendation module.
- Hospitals & Clinics — institutions that could integrate or reference the platform as a triage/education aid ahead of an in-person visit.
- System Administrators — the technical team responsible for maintaining the AI providers, monitoring API quotas (Groq/Gemini), managing the vector database, and ensuring the safety-validation layer stays effective.

3. Project Overview

3.1 Technology Stack

Layer	Technology
Frontend	Next.js 14 (App Router), TypeScript, Tailwind CSS
Backend	FastAPI, SQLAlchemy (SQLite → Postgres planned)
Orchestration	LangChain LCEL → migrating to LangGraph
Vector store	Pinecone serverless (AWS us-east-1)
Primary LLM	Llama 3.3 70B via Groq
Vision LLM	Gemini 3.5 Flash (fallback Gemini 2.5 Flash / Pro)
Embeddings	BAAI/bge-large-en-v1.5
Auth	JWT + Google OAuth

3.2 Project Workflow

Following the phased structure of a data-science project (definition → research → data/EDA → PoC → deployment → evaluation), here is where Medora sits today, phase by phase:

Phase	Status	Notes
1. Definition & use case	Done	Clinical assistant for patients; NIH MedlinePlus + curated textbooks as the knowledge base; Egypt-specific doctor referral as a concrete end-user feature.
2. Research & problem understanding	Done	Data pipeline (MedlinePlus alphabet sweep + Docling-parsed textbooks) established; RAG vs. fine-tuning decision made in favor of RAG + curated sources.
3. Data preparation / EDA	Done	4,822 MedlinePlus chunks + 9,513 textbook vectors in Pinecone (medical-assistant index, medical_textbooks_base namespace), 1024-dim BGE embeddings, cosine similarity.
4. Proof-of-concept model	Done	Working chat pipeline (RAG + Llama 3.3 70B) and vision pipeline (Gemini) both functional end-to-end.
5. Deployed PoC as a service	In progress	FastAPI backend + Next.js frontend both running; a full backend audit was completed before starting the next architectural step (see Section 9).
6. Evaluation & iteration	Ongoing	Mid-refactor: migrating to a LangGraph orchestrator, wiring disconnected clinical utilities, replacing mocked OCR, activating the safety/policy layer.

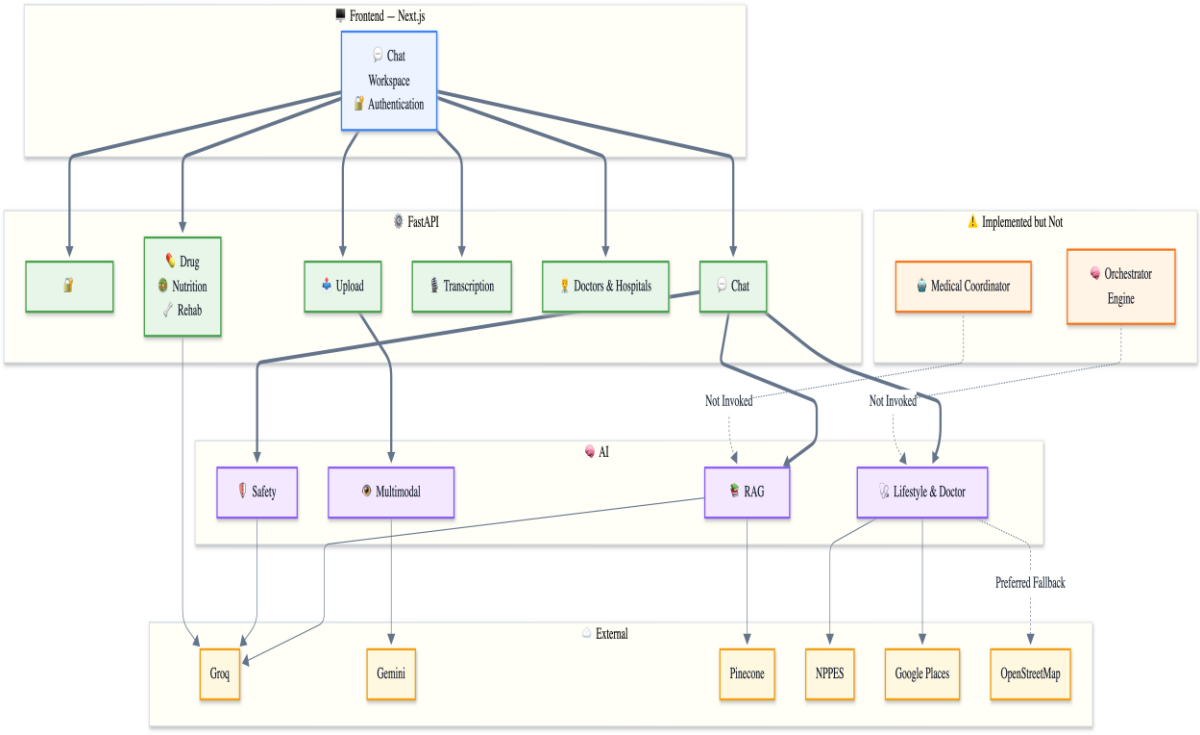
Current sprint focus (per the completed audit)

- Migrate /chat from direct function calls to a LangGraph graph with an LLM orchestrator node (llama-3.3-70b-versatile) making the routing decision.
- Wire the currently-disconnected clinical utilities (medication normalizer, entity resolver, evidence ranker, guideline lookup, timeline builder) into the live pipeline.
- Replace mocked OCR providers (PaddleOCR/EasyOCR) with real extraction.
- Activate the safety/policy layer so unsafe_request_handler actually runs before pipeline execution, and policy_engine does real evaluation instead of always returning "allowed."
- Add the drug-RAG, doctor-recommendation-RAG, and wound-vision pipelines that are currently only stubbed as Pipeline enum members.

4. System Architecture — Full Diagrams

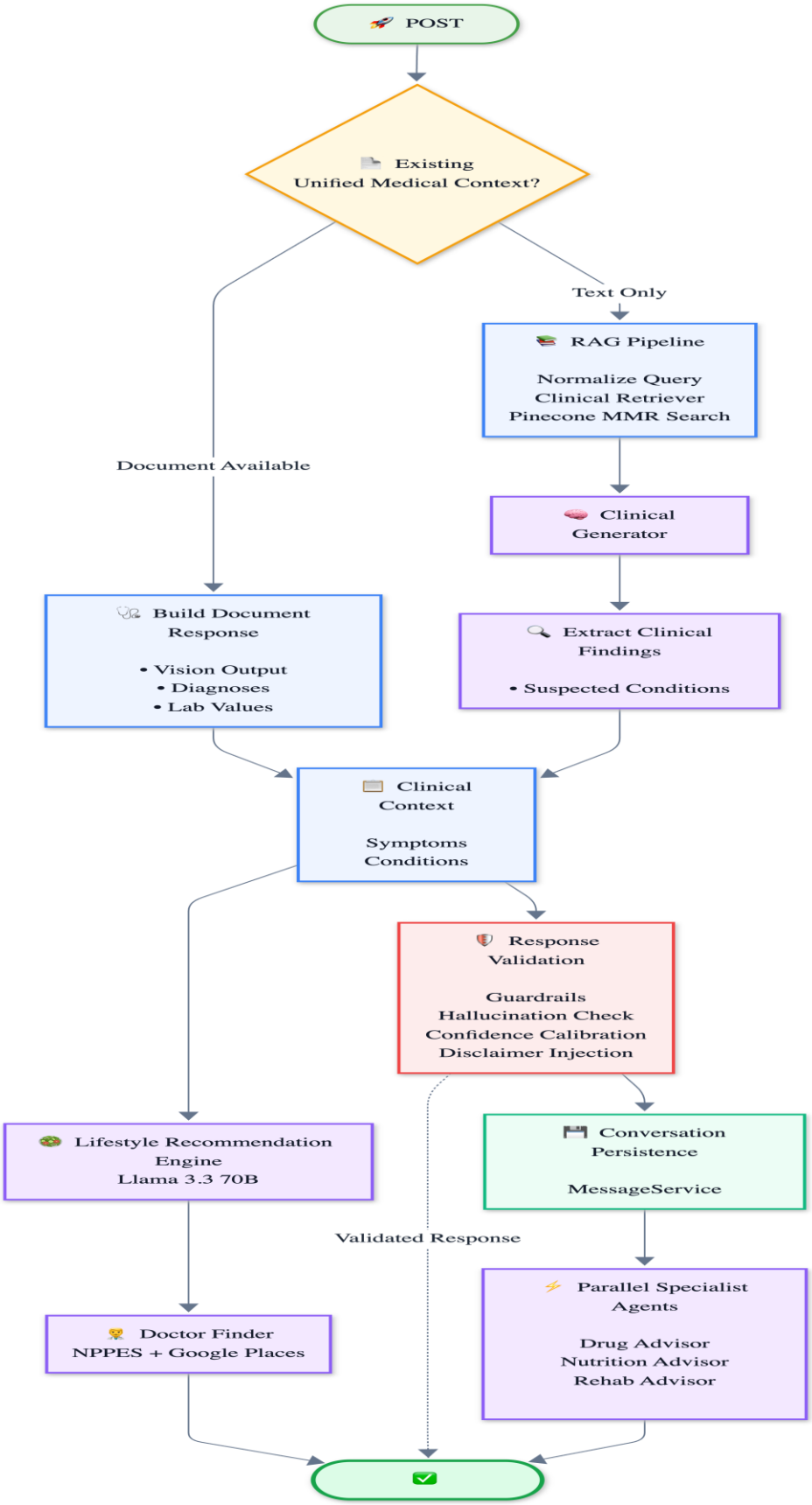
This section gives the complete, end-to-end map of the application — every pipeline, live or scaffolded. Each diagram is labeled with its current status so nobody mistakes an orphaned graph for a live one.

4.1 High-level system map



[Fig 4.1 — Full system map. Boxes marked LIVE are wired into a real FastAPI route today.]

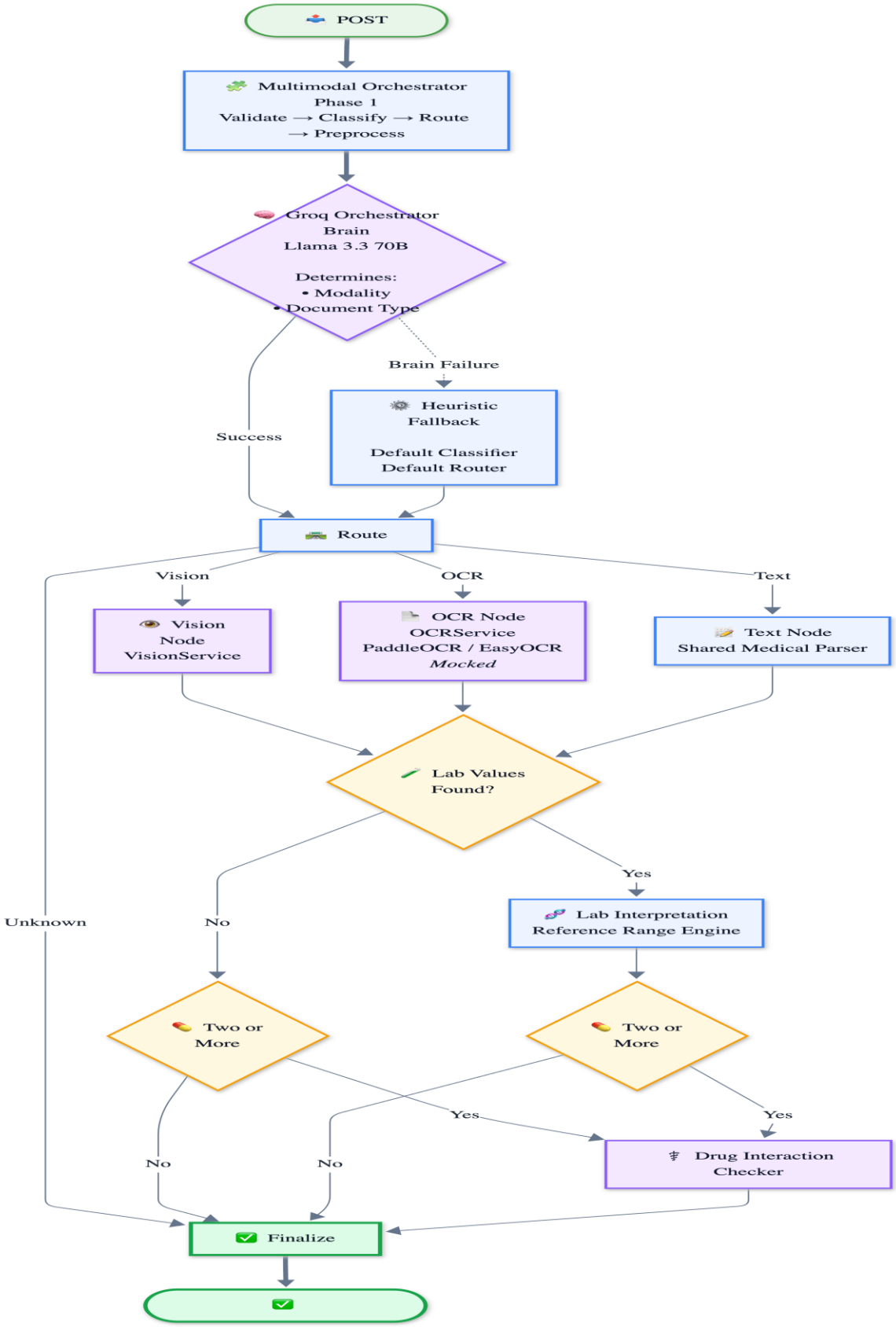
4.2 chat pipeline (live, current imperative implementation)



[Fig 4.2 — LIVE. Exact code path in app/api/chat.py today. When unified_context is present, Pinecone/RAG is bypassed entirely (documented architecture debt, not a bug).]

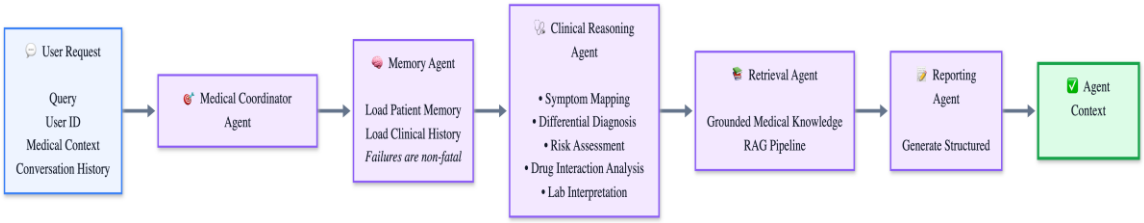
4.3 upload pipeline — Multimodal LangGraph (live)

This is the one LangGraph that is actually compiled and invoked in production today.



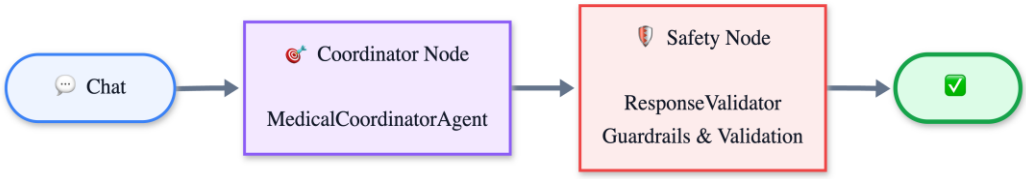
[Fig 4.3 — LIVE. Compiled via get_multimodal_graph() (app/ai/graph/multimodal_builder.py), invoked directly in app/api/upload.py.]

4.4 Agent Layer — MedicalCoordinatorAgent (implemented, currently orphaned)



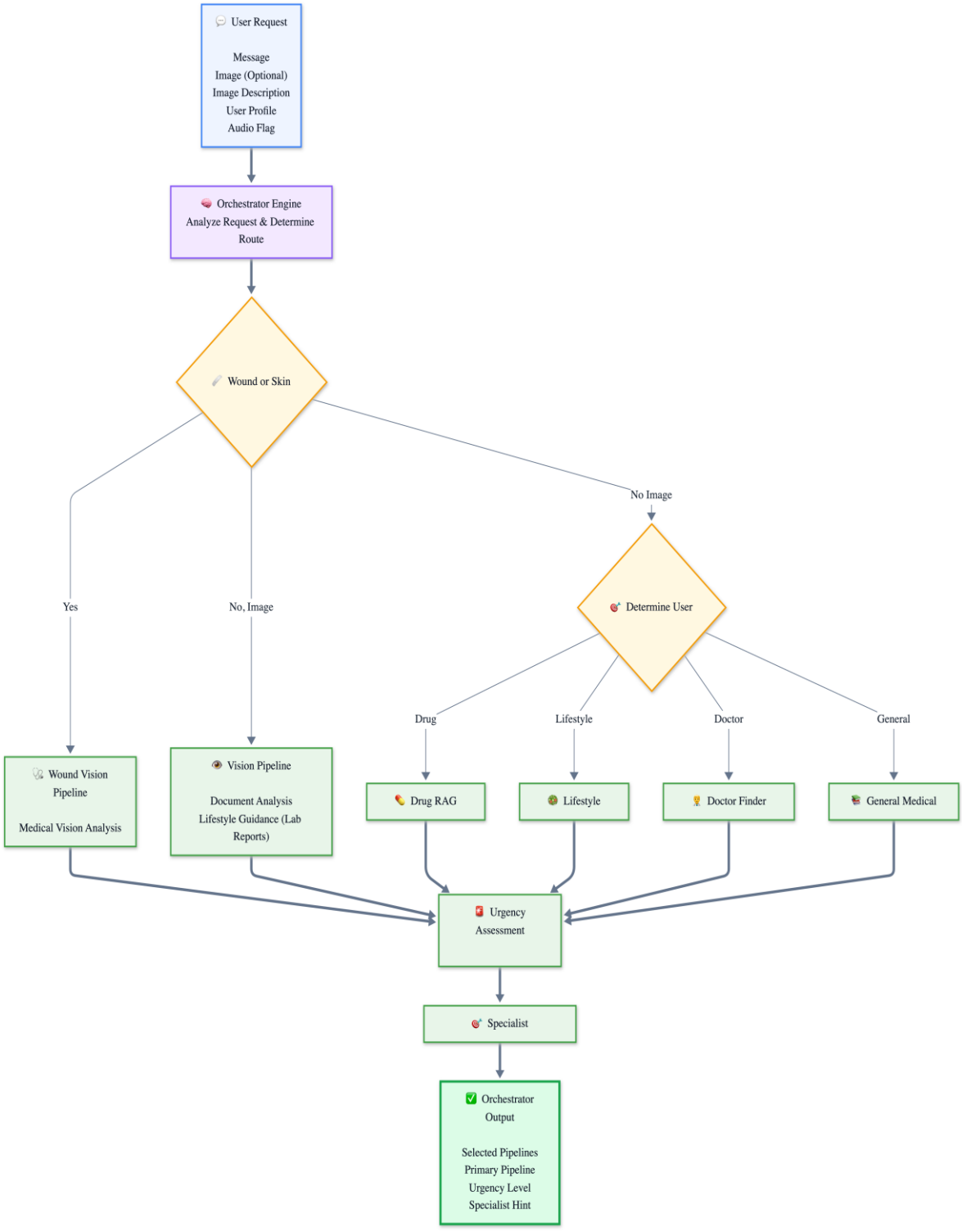
[Fig 4.4 — Implemented and independently testable, but no FastAPI route currently calls into it.]

There is also a two-node LangGraph wrapper around this exact pipeline (app/ai/graph/builder.py → build_medical_graph()):



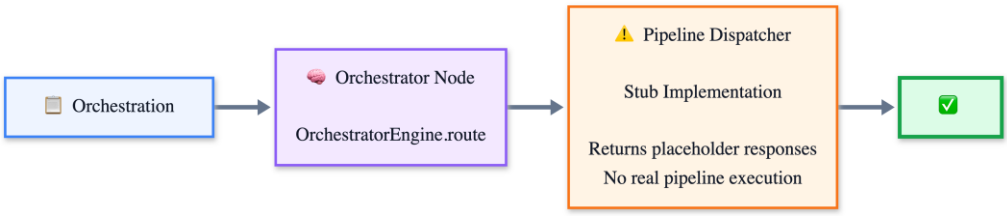
[Fig 4.4b — Neither the agent pipeline nor this graph is called by any FastAPI route today; /chat still calls the clinical/retrieval functions directly. This is the design the LangGraph migration is expected to promote to be the real routing layer.]

4.5 Rule-Based Orchestrator Engine + its LangGraph (implemented, currently orphaned)



[Fig 4.5 — Rule-based orchestration logic (no LLM).]

Its LangGraph wrapper (app/ai/graph/orchestration_builder.py):



[Fig 4.5b — ORPHANED, and its dispatcher node is a stub even in isolation. One of the pieces the LangGraph migration needs to either complete or replace.]

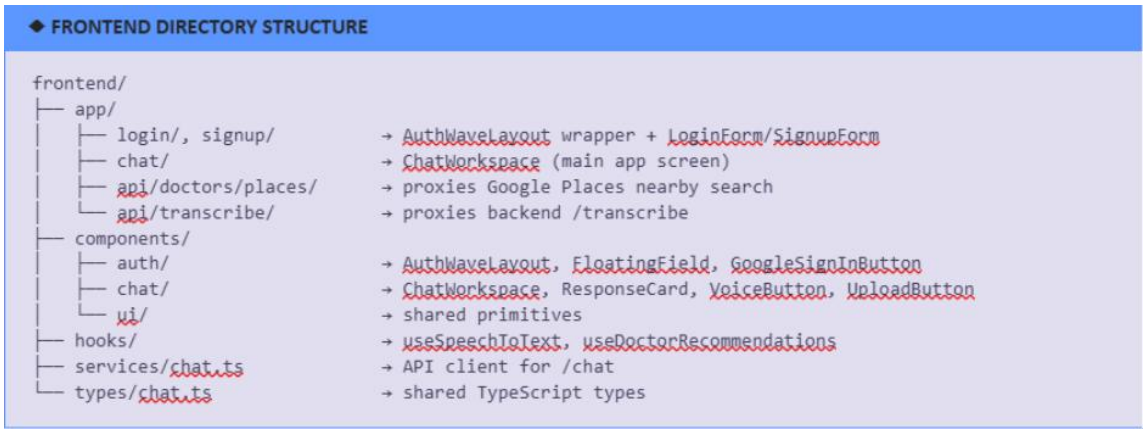
4.6 Safety Validation Pipeline (live, invoked from /chat and the orphaned safety_node)

[Fig 4.6 — LIVE for guardrails/hallucination/confidence/disclaimer. PolicyEngine always allows; UnsafeRequestHandler is fully implemented but never called before the pipeline runs — both flagged for the current sprint.]

5. Frontend

Stack: Next.js 14 (App Router), TypeScript, Tailwind CSS, JWT + Google One Tap OAuth.

5.1 Structure



FRONTEND DIRECTORY STRUCTURE:

- frontend/app/login/, signup/ → AuthWaveLayout wrapper + LoginForm/SignupForm
- frontend/app/chat/ → ChatWorkspace (main app screen)
- frontend/app/api/doctors/places/ → proxies Google Places nearby search
- frontend/app/api/transcribe/ → proxies backend /transcribe
- frontend/components/auth/ → AuthWaveLayout, FloatingField, GoogleSignInButton
- frontend/components/chat/ → ChatWorkspace, ResponseCard, VoiceButton, UploadButton
- frontend/components/ui/ → shared primitives
- frontend/hooks/ → useSpeechToText, useDoctorRecommendations
- frontend/services/chat.ts → API client for /chat
- frontend/types/chat.ts → shared TypeScript types

5.2 Design system

- Purple/violet theme (--primary: #d9c5ff, --primary-strong: #ae84ff), soft surfaces, rounded (24px) cards — defined in globals.css as CSS variables + Tailwind @theme inline mapping.
- Auth pages use AuthWaveLayout: animated SVG wave/blob background, glassmorphism card, floating-label pill inputs, gradient CTA buttons (from-[#8566FF] to-[#6f4ef2]).
- Fully responsive (single column mobile → dual column desktop), dark-mode-aware tokens already defined in globals.css (currently mirrored to the same light palette; not yet diverged).

5.3 Key interaction flows

- Chat: ChatWorkspace posts to /chat, renders ResponseCard with tabs for diagnosis / lifestyle / doctors / sources, supports markdown + tables.
- Voice input: useSpeechToText hook records audio → posts to the Next.js /api/transcribe route → proxies to backend /transcribe (Groq Whisper whisper-large-v3-turbo).
- Document upload: UploadButton posts the file to backend /upload; the returned unified_context is attached to the next /chat call so the document's findings ground the conversation.
- Doctor recommendations: useDoctorRecommendations hook can call either the backend's Egypt-aware search or the Next.js /api/doctors/places route (direct Google Places nearby search) depending on the flow.

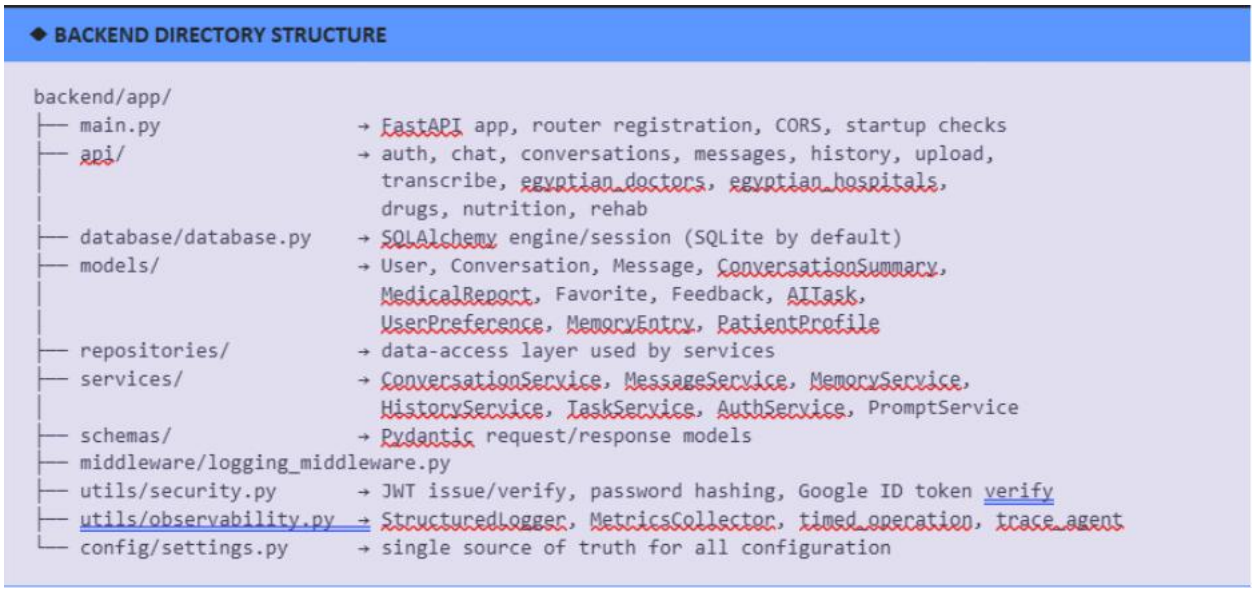
5.4 In-progress frontend work

- Auth page redesign: complete (dark-themed split layout, deep purple #0e0a1f background, Playfair Display serif headings, Outfit sans-serif body).
- Lab report UI redesign: in progress — full visual overhaul while preserving all existing content exactly; not yet finalized.

6. Backend — Core Infrastructure

Stack: FastAPI, SQLAlchemy ORM, SQLite (dev) with Postgres planned for the physician directory, JWT auth with Google OAuth verification.

6.1 Structure (infrastructure layer only — AI subsystems detailed in later sections)



BACKEND DIRECTORY STRUCTURE:

- backend/app/main.py → FastAPI app, router registration, CORS, startup checks
- backend/app/api/ → auth, chat, conversations, messages, history, upload, transcribe, egyptian_doctors, egyptian_hospitals, drugs, nutrition, rehab
- backend/app/database/database.py → SQLAlchemy engine/session (SQLite by default)
- backend/app/models/ → User, Conversation, Message, ConversationSummary, MedicalReport, Favorite, Feedback, AITask, UserPreference, MemoryEntry, PatientProfile
- backend/app/repositories/ → data-access layer used by services
- backend/app/services/ → ConversationService, MessageService, MemoryService, HistoryService, TaskService, AuthService, PromptService
- backend/app/schemas/ → Pydantic request/response models
- backend/app/middleware/logging_middleware.py
- backend/app/utils/security.py → JWT issue/verify, password hashing, Google ID token verify
- backend/app/utils/observability.py → StructuredLogger, MetricsCollector, timed_operation, trace_agent
- backend/app/config/settings.py → single source of truth for all configuration

6.2 Authentication

- POST /auth/signup, /auth/login — email/password with bcrypt hashing (passlib) and JWT (python-jose, HS256, 60 min expiry by default).
- POST /auth/google — verifies a Google ID token (google-auth), checks issuer allow-list and email verification, creates the user on first login with a random placeholder password.
- GET /auth/me — returns current user from Authorization: Bearer token.
- /chat supports optional authentication (get_current_user_optional) — anonymous chat works, but conversation persistence, memory, and history require a logged-in user.

6.3 Conversation & message persistence

Conversation (soft-deletable via status), Message (stores provider/model/execution_time/token_usage/citations/attachments/workflow/metadata as JSON), ConversationSummary (auto-generated when message count crosses MEMORY_CONVERSATION_LIMIT), MemoryEntry (typed longitudinal facts: condition, allergy, medication, preference), UserPreference (language, response style, preferred provider/model).

MemoryService.inject_memory() assembles user preferences + persistent memory + conversation summary + recent messages into the exact structured shape PromptBuilder.build_chat_prompt() expects, in a fixed order (system → preferences → persistent memory → summary → retrieved context → formatting instructions, with recent messages and the current question appended around it).

HistoryService supports full-text-ish search (LIKE) across conversation titles/message content, favoriting messages, and per-message feedback (1–5 rating + comment).

6.4 Configuration (app/config/settings.py)

Single Settings object (pydantic-settings, .env-driven) covering: API keys (Groq/Gemini/Pinecone/Google Places), model names per task (MODEL_CHAT, MODEL_VISION, MODEL_VISION_FALLBACK, MODEL_DOCUMENT, MODEL_REWRITE, MODEL_OCR, MODEL_EMBEDDING), provider-per-task mapping, Pinecone index/namespace, timeouts and token budgets per subsystem, OCR retry policy, upload constraints, safety toggles (SAFETY_ENABLED, SAFETY_BLOCK_UNSAFE_REQUESTS, etc.), feature flags per pipeline, and observability/logging config. Production mode enforces that GROQ_API_KEY, PINECONE_API_KEY, and DATABASE_URL are actually set (validator raises otherwise).

6.5 Provider abstraction layer

- BaseAIProvider / BaseChatProvider / BaseEmbeddingProvider define the contract; GroqProvider and GeminiProvider implement it.
- ProviderFactory + module-level get_provider() / get_default_llm() / get_default_embeddings() give every subsystem a single, cached way to obtain a model client without hardcoding SDK calls — this is what lets the vision pipeline and the chat pipeline swap models purely through settings.py without touching call sites.
- ModelRegistry (app/ai/providers/model_registry.py) is a discoverable catalogue of every registered model (name, provider, type, context length, description) — used for introspection/tooling, not for routing decisions itself.

6.6 Observability

app/utils/observability.py provides a JSON-structured logger (StructuredLogger), an in-memory MetricsCollector (counters + histograms, meant to be swapped for Prometheus/Datadog in production), a timed_operation context manager, and a trace_agent decorator that auto-instruments any BaseAgent.run() implementation with call/success/failure counters and duration histograms.

7. Vision Pipeline (Detailed)

This is the fully-owned, fully-documented subsystem for this hand-off. It is the path taken by every uploaded image or PDF that the multimodal orchestrator routes to ProcessorType.VISION (which, by design, is almost everything — see §7.5).

7.1 Models used

Role	Model	Provider	Notes
Primary VLM	Gemini 3.5 Flash (gemini-3.5-flash)	Google Gemini	settings.MODEL_VISION. Ingests images and PDFs natively — no separate OCR step needed for the primary path.
Fallback VLM	Gemini 2.5 Flash (gemini-2.5-flash)	Google Gemini	settings.MODEL_VISION_FALLBACK. Triggered on 429/5xx, rate limit, quota, timeout, "overloaded", or model-decommissioned errors.
Document/structured-extraction model	Gemini 2.5 Flash → Groq Llama 3.3 70B fallback	Gemini → Groq	Used by SharedMedicalParser to turn the vision model's free-text narrative into structured JSON.
Registry-listed alternative	gemini-2.5-pro	Gemini	Registered as a general-purpose Gemini vision option; not currently wired as active default or fallback.
Groq vision fallback (registered, not wired)	llama-4-scout	Groq	Present in ModelRegistry/ModelRouter as a theoretical Groq-side vision fallback; not implemented in VisionProvider's actual fallback chain.

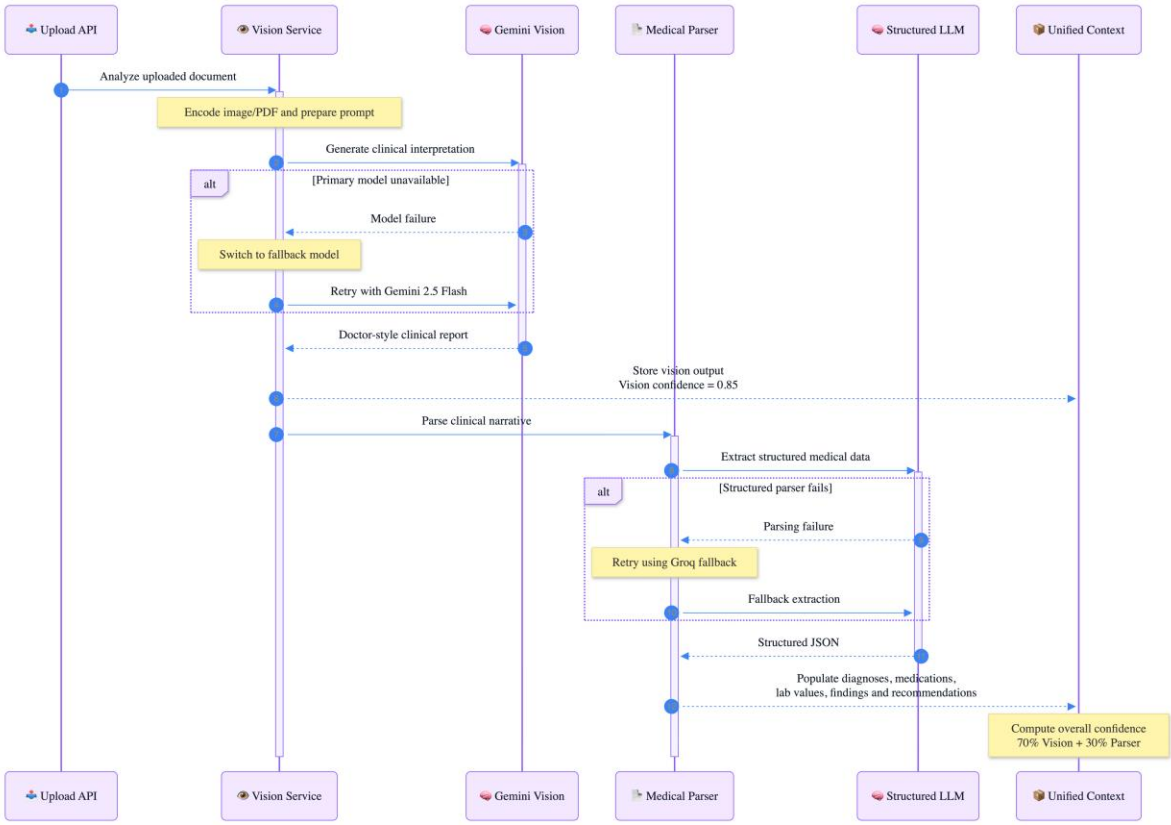
7.2 Why Vision-first, not OCR-first

DefaultRouter deliberately routes both images and PDFs to Vision, not OCR: "PDFs to the Vision model (Gemini). Gemini ingests PDFs natively and produces a far richer clinical interpretation (lab-value comparison, drug extraction, doctor-grade summary) than raw OCR text extraction. OCR remains available as an internal fallback inside the vision service if needed." — app/ai/multimodal/router.py

The GroqOrchestratorBrain (the LLM that makes the classify+route decision in Phase 1 of /upload) is given the same guidance directly in its system prompt: route to VISION for "images (skin, wounds, x-ray, MRI, CT, ultrasound, eye) and PDFs/documents (lab reports, prescriptions, referrals, discharge summaries)", and reserve OCR "only when the file is a scanned document that genuinely needs text extraction."

7.3 The two-call vision architecture

Every document that reaches VisionService.process() goes through two separate LLM calls, not one:



[Fig 7.3 — The vision two-call sequence: narrative generation, then structured extraction.]

Critical design guarantee: if the second call (structured parsing) fails for any reason, the raw vision output is never lost — VisionService.process() catches the parser exception, logs it, sets parser_confidence = 0.0, and still returns the context with the full doctor-voice narrative intact. The frontend/chat layer is written to fall back to showing that raw narrative if structured fields are empty (see _build_document_response in app/api/chat.py).

7.4 The vision prompt design (app/ai/prompts/vision_prompts.py)

This is a deliberately engineered prompt, not a generic "describe this image" call:

- VISION_SYSTEM_INSTRUCTION casts the model as a senior board-certified internist speaking directly to the patient — calm, complete, never a one-line summary, reads every value on the document (including normal ones), reports exact figures with units, uses the document's own printed reference range when present (falling back to "typical adult range" only when it isn't), explicitly flags anything unreadable rather than guessing.
- VISION_ANALYSIS_PROMPT is a 4-step instruction: (1) identify the document type and extract patient/date/clinic/doctor metadata; (2) if lab results are present, emit a Markdown table (Test | Result | Unit | Normal Range | Status | What this means) for every single value, then "Reassuring findings" and "Findings worth discussing with your doctor" sections; (3) if a prescription is present, for every drug emit dose/route/frequency/duration/likely indication/warnings, followed by a "how to take your medicines safely" summary; (4) close with a 2–4 sentence "Doctor's overall impression" plus a one-line educational disclaimer.
- The prompt explicitly tells the model to use its full token budget rather than stopping early — this is why AI_MAX_TOKENS_VISION is set generously (16,384) in settings, since Gemini's "thinking" models share that budget between internal reasoning and the visible answer.
- A legacy MEDICAL_PARSER_PROMPT constant is kept only for backward compatibility with older callers and is explicitly documented as never allowed to overwrite the rich narrative output.

7.5 Resilience characteristics

Concern	Mechanism
Oversized images	DefaultPreprocessor (app/ai/multimodal/preprocessing.py) downsizes any image over 30M pixels or 6000px on the long edge before it reaches the vision model, preserving format (JPEG/PNG/WEBP) and handling RGBA→RGB flattening for JPEG output.
Transient provider failure	tenacity-based retry: 3 attempts, exponential backoff (2–10s) around every generate() call.
Hard timeout	min(AI_TIMEOUT_VISION, AI_MAX_TIMEOUT_VISION) — configurable soft timeout capped by an absolute ceiling (default 45s soft / 90s hard).
Model-level failure (429/5xx/quota/decommissioned/etc.)	Automatic, one-time provider/model swap to the fallback (gemini-2.5-flash), then retried with the same retry policy.
Structured-parse failure	Non-fatal — raw vision narrative is preserved; parser_confidence set to 0 and a warning logged instead of raising.
Confidence reporting	Fixed heuristic estimates today (vision_confidence = 0.85, parser_confidence = 0.95 on success) rather than a model-reported score — flagged here for anyone later wiring in real confidence signals.

7.6 Known gaps in this subsystem (for the LangGraph migration)

- Vision confidence values are hardcoded constants, not derived from the model call itself.
- The Groq fallback (llama-4-scout) is registered in the model catalogue but not actually implemented as a vision-provider fallback path — only Gemini↔Gemini fallback exists in code.
- Wound-specific vision (the planned wound_vision pipeline referenced in app/ai/orchestrator/pipelines.py) is not yet a distinct code path — wound images currently still flow through the same generic VisionProvider/VISION_ANALYSIS_PROMPT, which is written for lab reports and prescriptions, not dermatological assessment. This is one of the "new files needed" call-outs for the orchestrator migration (a dedicated wound_vision_node) — see Section 8.7 for the proposed module addressing this gap.

8.

8.1 Retrieval / RAG Pipeline — Clinical Reasoning Agent (Function Calling)

Purpose

The Clinical Reasoning Agent is the decision-making core of the retrieval pipeline. When a user asks a medical question, the agent determines the best way to answer it: by consulting the organization's internal medical knowledge base first, and by searching the live web only if the internal library does not contain a sufficient answer. This mirrors how a clinician would first consult trusted reference material before turning to broader external research, and directly implements the RAG layer referenced elsewhere in this document (ClinicalGenerator + ClinicalRetriever, Section 8.6) using an agentic, function-calling design rather than a fixed retrieval script.

Models Used / Architecture

The agent uses function calling (tool use): the underlying language model is given two callable tools and a set of rules, and it independently decides — at each step — which tool to use and when it has enough information to answer.

- Tool 1 — Internal Knowledge Search: queries a vector database (Pinecone) containing curated, pre-loaded medical textbooks. This is always attempted first for every medical question.
- Tool 2 — External Web Search: is triggered only when the internal search tool returns no relevant result, or when the retrieved content does not actually answer the question asked. This tool performs a live search and synthesizes a factual, evidence-based summary.

This tool-first approach prevents the model from answering purely from memorized training data, which reduces the risk of outdated or fabricated ("hallucinated") medical information.

Data Flow / Diagram

Step	Action	Outcome
1	User submits a medical question	Agent classifies the question and begins reasoning
2	Agent calls the Internal Knowledge Search tool	Returns relevant textbook excerpts, or "NOT_FOUND"
3	If internal result is sufficient	Agent drafts the final answer and cites internal sources
4	If internal result is insufficient / not found	Agent calls the External Web Search tool
5	Agent synthesizes the final answer	Response is labeled Internal or External source accordingly

Knowledge base data collection & preparation: the internal knowledge base that the agent searches first is built from a combination of curated medical websites and specialized medical textbooks, rather than being generated by the AI model itself, ensuring the foundation of the assistant's answers is grounded in vetted, human-authored content.

- Website-sourced data: relevant medical information collected from specialized medical websites covering diseases, conditions, and general clinical guidance.
- Book-sourced data: additional content collected from specialized medical textbooks, adding depth and clinical detail beyond what is available on public websites.

After collection, the raw data was structured and enriched before being loaded into the knowledge base. As part of this preparation step, two additional columns were added to the dataset for each disease or condition: Symptoms (a dedicated column listing key symptoms) and Recovery / Pain Management (a dedicated column describing how to recover from the condition or reduce and manage associated pain). This structured format allows the retrieval step to surface not just general descriptions, but the specific symptom and recovery information needed to answer a user's question directly.

Multilingual support: the agent automatically detects whether the user's question is written in Arabic or English and responds entirely in that language, without mixing languages. Internally, non-English queries are translated before searching the internal knowledge base (since the source textbooks are in English), but the final answer is always delivered in the user's original language.

Current Status

Functional as an agentic, function-calling retrieval layer with an internal-first / external-fallback policy, a curated and enriched knowledge base (websites + textbooks, with dedicated symptom and recovery columns), and Arabic/English multilingual handling. This complements the RAG scaffolding already listed in this section's original file inventory (rag_pipeline.py, retrievers.py, generators.py, and the stub abstractions embedder.py, query_rewriter.py, reranker.py, hybrid_search.py, context_formatter.py, retriever.py) by providing the agent-level decision logic that decides how those retrieval building blocks should be invoked.

Known Issues / Next Steps

- The agent is instructed never to answer purely from its own memory — it must use one of the two tools before responding; this constraint should be verified with adversarial test prompts.
- The internal knowledge base is treated as the authoritative source; external search is explicitly a secondary, fallback mechanism — reconcile this policy with the existing ClinicalRetriever/ClinicalGenerator RAG path used in the live /chat pipeline (Fig 4.2) so there is a single source of truth for retrieval behavior.
- Every answer discloses whether it came from the internal library or a live web search, supporting transparency and auditability — this labeling should be surfaced in the ResponseCard "sources" tab on the frontend (Section 5.3).
- The external web search tool is explicitly flagged as not suitable for use with real, identifiable patient data (not HIPAA-compliant), and is intended for general medical research queries only.
- A resilience mechanism is built in: if the primary external search service is unavailable, the system automatically falls back to the core language model's general knowledge and clearly labels the answer as such.
- Reconcile this module with the disconnected retrieval abstractions noted in Section 9.2 (embedder, query rewriter, reranker, hybrid search, context formatter) — determine which, if any, should be adopted as internal building blocks for the Internal Knowledge Search tool.

8.2 Medical Image Analysis (AI Vision) — Business Perspective

A user uploads a photo of a visible condition (a wound, a skin rash, a visible injury...), and the system returns a structured visual assessment: one primary likely diagnosis + ranked alternatives + basic care tips + a clear recommended next step (e.g. "see a dermatologist within two weeks"). The goal isn't to replace a doctor — it's to shorten the "worry and hesitation" period a user goes through before deciding to seek care, and give them organized information they can act on immediately.

Architecture & Models Used

This subsystem employs a multi-tiered resilience approach utilizing modern large vision models (LVMs) to guarantee high service uptime:

Tier	Component / Model	Provider	Role
Primary	meta-llama/llama-4-scout-17b-16e-	Groq	Executes deep visual analysis using advanced vision-language processing pipelines based on a

	instruct meta-llama/llama-4-maverick-17b-128e-instruct		specialized diagnostics prompt. Checked sequentially in priority order.
Fallback	Salesforce/blip-image-captioning-large	Hugging Face Inference API	Free, 100% available image captioning fallback triggered instantly if all primary Groq endpoints time out or fail.

How this works technically (without getting too deep)

- The image is encoded to base64 and sent to a vision-capable AI model.
- There's a **priority order between two models**: if the first model is unavailable or hits a temporary issue, the system automatically retries with the next one — the user never experiences a visible failure, which makes the service feel "always on" even when a provider has an outage.
- If both fail (a rare edge case), there's a **third line of defense (fallback)**: a simple, free model returns a general description of the image instead of an empty error message. So the user experience is covered even in the worst-case scenario.

Why the response is structured, not just a list of possibilities

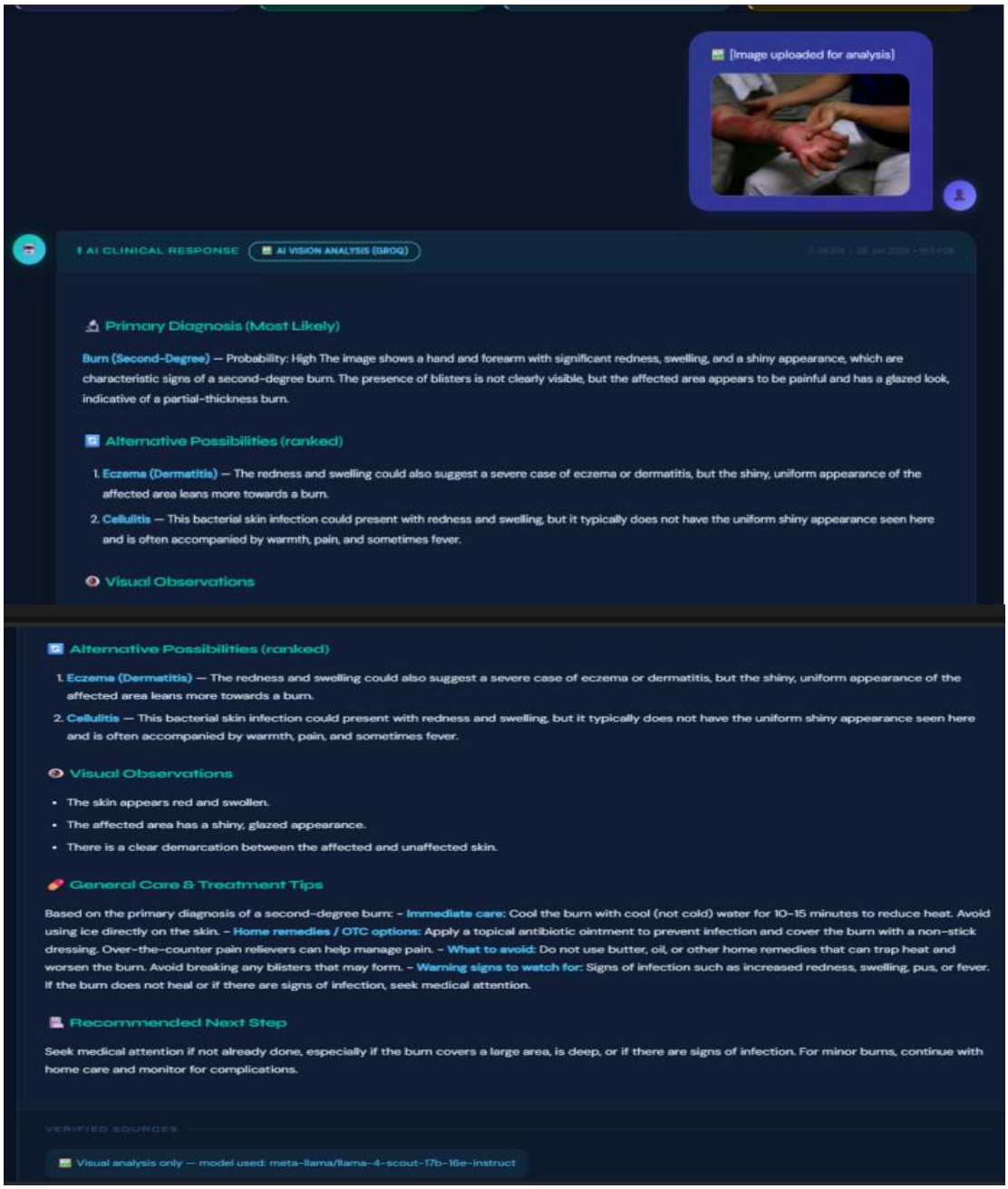
This point matters from a business standpoint: any system that throws 5 equally-weighted possibilities at a user leaves them confused and unsure what to do next. That's why the response follows a fixed structure:

1. **One decisive primary diagnosis** — with a confidence level (High/Moderate) and the visual reasoning behind it.
2. **Ranked alternatives (2–3)** — from most to least likely, not a random list.
3. **Visual observations** — color, shape, size, distribution.
4. **Basic care tips** — what to do now, what to avoid, and when it becomes urgent.
5. **A recommended next step** — one clear, actionable recommendation.
   It closes with a clear disclaimer that this is an AI-generated visual impression, not a final medical diagnosis — which protects the product both legally and ethically.

Business value

- **Reduced friction**: instead of waiting for an appointment to find out "is this minor or do I need to act fast," the user gets an immediate initial read.
- **Retention**: a visual feature like this differentiates the product from a plain text-based medical chatbot, giving users a reason to keep coming back whenever a new symptom appears.
- **Clear conversion path**: the final recommendation (e.g. "see a dermatologist") can later be linked to an in-app doctor-booking feature — so this isn't just a diagnostic feature, it's also a gateway to other revenue-generating services.
- **Lower support cost**: instead of a full team fielding "is this dangerous or not?" questions, the AI filters out a large share of them upfront.

Placeholder for the test screenshot



8.3 Lifestyle Recommendation Module

Purpose

The Lifestyle tab is intended to surface non-pharmacological guidance — diet, activity, sleep, monitoring, and self-care instructions — that the model derives from the same grounded context used for the clinical/diagnostic portion of the answer. Functionally, it re-uses the chat pipeline's output rather than introducing a second retrieval pass or a separate lifestyle-specific knowledge base; the MedlinePlus corpus already contains consumer-oriented health-topic text (symptoms, self-care, and prevention sections) that naturally supports this kind of guidance, and the textbook sources (particularly Symptoms to Diagnosis) provide supporting clinical rationale.

Models Used

The chat model (Llama 3.3 70B via Groq) is prompted to produce a structured response; system/prompt-engineering constraints require every factual claim to map to a retrieved passage, and

the response is expected to separate clinical assessment from behavioral/self-care guidance. The retrieval stage embeds the query with a BAAI/bge-large-en-v1.5 model (1024 dimensions) and performs Maximal Marginal Relevance (MMR) search against the Pinecone index (k = 4, fetch_k = 10), returning passages drawn from the NIH MedlinePlus corpus (4,822 chunks over 772 topics) and four clinical textbooks (9,513 vectors: Gray's Anatomy for Students, Mosby's Diagnostic & Laboratory Test Reference, Learning Radiology, and Symptoms to Diagnosis).

Data Flow / Diagram

- User input (text, transcribed voice, or parsed document) is combined with retrieved MedlinePlus/textbook passages into a single grounded context. - The chat model (Llama 3.3 70B via Groq) is prompted to produce a structured response; system/prompt-engineering constraints require every factual claim to map to a retrieved passage, and the response is expected to separate clinical assessment from behavioral/self-care guidance. - The frontend's ChatWorkspace/ResponseCard components route the behavioral-guidance portion of the completion into the Lifestyle tab, alongside a Sources tab that lists the citations used, giving the person a way to audit the guidance against MedlinePlus or textbook text. Coupling lifestyle guidance to the same citation-constrained pipeline as the clinical answer is a reasonable mitigation against a known failure mode of consumer health chatbots: LLMs asked open-endedly for "lifestyle advice" tend to produce generic, occasionally overreaching recommendations (e.g., specific caloric targets or exercise prescriptions) that are not warranted by the input. By construction, MedCortex's refusal-on-insufficient-context behavior should apply equally to this tab, so that lifestyle guidance is either traceable to a MedlinePlus topic/textbook passage or withheld.

Interface Contract (Inferred)

Trigger: Any /chat completion whose grounded context contains self-care, prevention, or behavioral content. Input: User query + retrieved MedlinePlus/textbook passages + optional structured vision/lab data. Output: Free-text guidance segment, rendered in the Lifestyle tab, cross-referenced with the Sources tab. Grounding rule: Same no-hallucination / refuse-if-unsupported policy as the rest of the answer. Failure mode: Empty or generic tab if retrieval returns no behaviorally relevant passages.

Current Status

This is a description of observed interface behavior, based on the documented /chat endpoint and the tabbed rendering behavior, rather than a behavior independently verified against the live system, since the exact prompt template used to elicit and separate the Lifestyle section was not exposed in the inspected documentation.

Known Issues / Next Steps

- No drug-interaction checker yet exists, which would be a natural complement to lifestyle guidance involving medication timing. - No symptom-checker / differential-diagnosis scoring module, which could improve the specificity of lifestyle guidance. - No multi-language support (e.g., Arabic) yet, which limits accessibility of lifestyle guidance for the platform's primary regional audience. - No stated integration with electronic health record (EHR) systems, which would allow lifestyle guidance to be personalized against a verified medical history rather than a single conversation's context. - Introduce a lightweight structured-output contract (e.g., a JSON schema with explicit lifestyle, referral, and citation fields) between the chat model and the frontend, replacing free-text tab-splitting with a verifiable, testable data contract. - Add confidence/coverage indicators to the Lifestyle tab so users can see when guidance is well-supported by multiple sources versus a single thin passage. - Add Arabic-language support to the lifestyle text, consistent with the project's stated multi-language roadmap and its Egypt-specific target audience.

8.4 Doctor / Physician Recommendation Systems

Purpose

This module answers the question "which doctor should this patient see, and where?" by combining semantic retrieval with real geographic reasoning. Rather than returning doctors ranked by rating alone, the system resolves the patient's location from their own words or, when no location is mentioned, from their real-time device location via the browser geolocation API.

It constrains the candidate pool to that geographic area using verified Egyptian administrative boundaries, and ranks the candidates by a blended relevance-and-quality signal before an LLM turns the ranked list into a natural-language recommendation.

Data Source

The underlying doctor and clinic dataset was built via a custom scraping pipeline against Google Maps, capturing structured fields per listing — name, category/specialty, address, coordinates, phone number, aggregate rating, and review count — and indexed into a dedicated Pinecone namespace (clinics_doctors_egypt) separate from the medical-knowledge namespaces used elsewhere in the RAG pipeline (Section 8.1), in line with the architectural rule that structured directory-style data should never be mixed into the clinical knowledge index.

Models / Components Used

Role	Model / Component	Notes
Location extraction	llama-3.3-70b-versatile (Groq)	Strict JSON-only extraction of the raw place name/phrase from the user's query.
Query embedding	BAAI/bge-large-en-v1.5	Same embedding model used across the rest of the platform, ensuring a single embedding space.
Geocoding	Local GeoJSON boundary file (Egyptian governorates + localities)	Matched via shapely polygon containment and fuzzy string matching. No external geocoding API — fully self-hosted and offline-capable.
Response generation	llama-3.3-70b-versatile (Groq)	Structured "MedFinder AI" persona prompt that turns the ranked candidate list into a patient-facing recommendation.
Data collection	Apify (Google Maps Scraper)	Used to extract structured doctor/clinic listings from Google Maps at scale

Location Resolution

A dedicated location-extraction prompt pulls the exact geographic phrase from the user's message (governorate, district, or neighborhood). That phrase is then geocoded entirely offline against a local GeoJSON file of Egyptian administrative boundaries: an exact or fuzzy match against a governorate (admin_level=4) is attempted first, and if sub-location tokens remain, they are matched against localities whose geometry actually intersects that governorate's polygon — preventing a locality name from incorrectly resolving to the wrong region.

If a location is named but cannot be matched to any known boundary, the search is intentionally not silently widened; it is reported as not found rather than falling back to a broader, less accurate search. If the user does not mention a location at all, the system dynamically invokes the browser Geolocation API to retrieve and utilize the user's live device coordinates, fully replacing development-time placeholders with automated real-time resolution.

Retrieval and Ranking

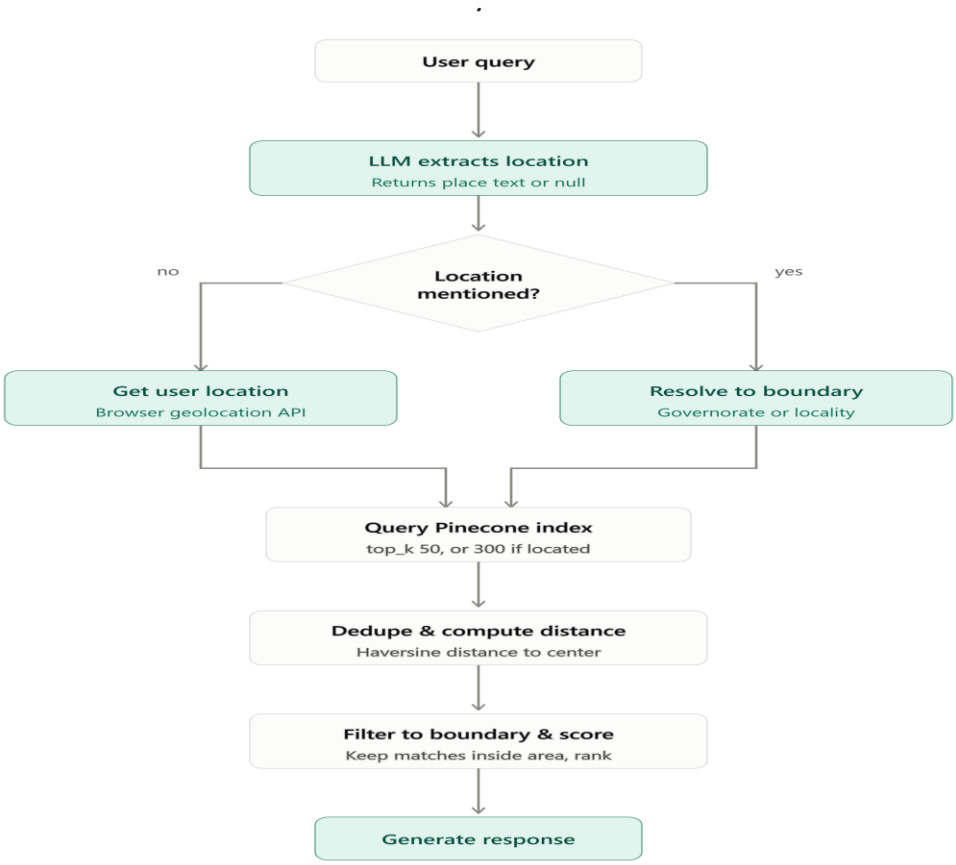
Once a reference point is established, a bounding box derived from either the matched boundary polygon or a distance-based radius is applied as a metadata pre-filter on the Pinecone query, and the top candidates are retrieved by semantic similarity to the user's query. Candidates are deduplicated, their distance from the reference point is computed via the Haversine formula, and they are grouped into concentric distance rings (20 km / 40 km / 60 km / beyond).

Every candidate is then scored on three normalized signals — vector similarity, a Bayesian-adjusted rating (which pulls low-review-count listings toward the dataset average rather than letting a handful of 5-star reviews dominate), and a log-scaled review-volume score —

combined into a single weighted quality score. The final ranking sorts first by whether the doctor falls within the resolved boundary, then by distance ring, then by quality score within each ring, so that proximity and administrative correctness always take precedence over raw relevance.

Response Generation

The ranked candidate list is passed as grounding context to a Groq-hosted llama-3.3-70b-versatile call using a dedicated "MedFinder AI" system prompt, which synthesizes the structured results into a warm, concierge-style natural-language recommendation rather than a raw list.



[Figure 8.5: End-to-End Execution Flow of the Doctor Recommendation Pipeline]

8.5 Hospital Recommendation System (OSM-based)

Purpose

This module extends the platform's location-aware recommendation capability to hospitals specifically, giving users the ability to find the nearest appropriate hospital rather than an individual physician. It complements the doctor-recommendation module above (Section 8.4) as a distinct feature: doctors are sourced from a curated Google Maps scrape indexed in Pinecone, while hospitals are sourced from a custom offline extraction pipeline built on the OpenStreetMap Egypt dataset.

Data Source

The hospital dataset was generated through a custom offline extraction pipeline built on the `egypt-latest.osm.pbf` OpenStreetMap dataset. Using the `osmium` library, the pipeline parsed the nationwide OSM extract and filtered all features tagged with `amenity=hospital`. For each hospital, the pipeline extracted its geographic coordinates and available metadata, then enriched the records with human-readable addresses and governorate information through reverse geocoding using `Nominatim`. The processed records were exported into a structured JSON dataset (`egypt_hospitals_full.json`) used by the hospital recommendation service. Because the source is OpenStreetMap rather than a commercial business directory, the dataset provides broad nationwide coverage, including many public hospitals that are not consistently represented in commercial mapping services.

Models / Components Used

Role	Model / Component	Notes
Data source	`egypt-latest.osm.pbf` (OpenStreetMap)	Nationwide offline geographic dataset downloaded from Geofabrik
Data extraction	`osmium`	Parses the PBF file and extracts all features tagged with `amenity=hospital`
Address enrichment	`Nominatim` Reverse Geocoding	Converts hospital coordinates into readable addresses and governorate names
Distance calculation	Haversine formula	Computes the distance between the user's location and each hospital

Data Flow

The hospital dataset is generated offline before deployment through the extraction pipeline described above and stored as a structured JSON dataset. At runtime, the user's real-time location is used as the reference point (no location-extraction-from-text step is required, since this feature performs a nearest-facility lookup rather than a free-text search). The distance between the user and every hospital record is computed using the Haversine formula, after which the hospitals are ranked in ascending order of distance and the nearest facilities are returned.

Current Status

Exposed via the `/egyptian-hospitals` endpoint, which serves recommendations from the preprocessed hospital dataset generated by the offline extraction pipeline. It is listed as LIVE in the system map (Fig 4.1) and in the backend directory structure (Section 6.1), functioning as a distinct, standalone feature from the doctor-recommendation module.

8.6 OCR Subsystem

Purpose

OCR is the fallback text-extraction path inside the multimodal pipeline — it exists for the narrow case where the upload orchestration brain (GroqOrchestratorBrain, Section 7.2) decides a file is a scanned document that genuinely needs plain text extraction, rather than something the Vision model (Gemini) can interpret directly with richer clinical understanding. Because Vision is deliberately preferred for almost everything (images and PDFs alike — see §7.2), OCR is the minority path, but it still needs to be a real, working engine rather than a placeholder before the system can be considered production-ready.

Architecture (already fully built — only the inference calls are mocked)

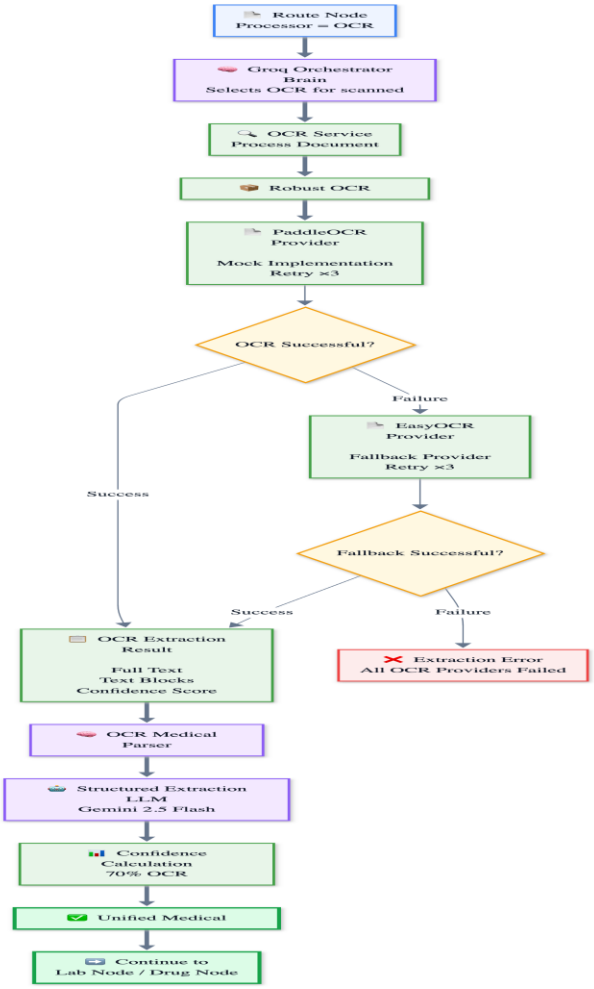
- BaseOCRProvider — abstract contract (provider_name, is_available(), extract_text()) implemented by two concrete providers.
- PaddleOCRProvider (primary) and EasyOCRProvider (fallback) — both implement the contract; is_available() checks whether the underlying package (paddleocr / easyocr) is actually importable in the environment.

- OCRProviderFactory — simple factory returning a provider instance by name string ("paddleocr" | "easyocr").
- RobustOCRExtractor — the resilience layer: tries the primary provider, then each fallback in order; every attempt is wrapped in a tenacity retry (3 attempts, exponential backoff 2–10s) and an asyncio.wait_for timeout (15s default). If a fallback provider succeeds, it appends a warning to the result noting the primary failed. If every provider fails, it raises a single consolidated ExtractionError.
- OCRMedicalParser (extends SharedMedicalParser) — stores the raw OCR text onto unified_context.ocr_output, then delegates to the exact same LLM-based structured-extraction step used by Vision (Section 7.3, call #2: Gemini 2.5 Flash → Groq Llama 3.3 70B fallback, same JSON schema).
- OCRService.process() — the orchestration entry point: extractor.extract() → parser.parse() → blends confidence as (ocr_confidence * 0.7) + (parser_confidence * 0.3), mirroring the exact blending formula used in VisionService (Section 7.3).

Models / Engines Used

Role	Engine	Status
Primary OCR engine	PaddleOCR (PP-OCR family)	MOCKED — provider returns a hardcoded string ("Amoxicillin 500mg, Diagnosis: Strep Throat"); real PaddleOCR inference not yet wired.
Fallback OCR engine	EasyOCR (CRAFT detector + CRNN recognizer)	MOCKED — provider returns a hardcoded string ("Patient John Doe. Blood Pressure 120/80."); real EasyOCR inference not yet wired.
Structured extraction (post-OCR)	Gemini 2.5 Flash → Groq Llama 3.3 70B fallback	LIVE — this half of the pipeline is real; it is the same SharedMedicalParser call used by Vision.

Data Flow / Diagram



[Fig 8.5 — OCR execution path. Structurally identical to the Vision path (Fig 7.3) after extraction — same parser, same schema, same confidence-blend formula — the only difference is what produces the raw text.]

Current Status

Flag: Both PaddleOCRProvider and EasyOCRProvider currently return hardcoded mocked output instead of running real inference. Every layer around them — retry/timeout/fallback logic, structured parsing, confidence blending, and downstream lab/drug enrichment — is real and already exercised end-to-end by the multimodal graph (Fig 4.3); only the two leaf extract_text() calls need to be swapped for genuine engine calls.

Known Issues / Next Steps

- Wire real PaddleOCR inference in PaddleOCRProvider.extract_text() (convert image_bytes → numpy array via cv2/PIL, call the PaddleOCR instance, map its output into OCRTextBlock entries with real per-block confidence instead of a single flat average).
- Wire real EasyOCR inference in EasyOCRProvider.extract_text() as the genuine fallback engine.
- Confirm GPU vs. CPU behavior and cold-start latency for both engines against the existing 15s per-attempt timeout — PaddleOCR/EasyOCR model loading on first call can be slow, so a warm-up step at app startup may be needed to avoid tripping the timeout on the very first request.
- For Arabic/RTL prescriptions and lab reports specifically, extract with PyMuPDF (fitz) rather than pdfplumber if any pre-OCR PDF text layer exists — pdfplumber reverses RTL character order, PyMuPDF handles Unicode RTL correctly (established pattern already used elsewhere in this project for Arabic PDF ingestion).
- Once real engines are in place, replace the fixed average_confidence estimate with the engine's genuine per-block confidence scores, and reconsider the fixed 0.7/0.3 blend weight against real data.

- Add an explicit unit test that forces the OCR path (bypassing the Vision-preferring orchestration brain) so this subsystem isn't only exercised incidentally.

8.7 LangGraph Orchestrator Migration (the central refactor)

Purpose

Today the application has three separate, non-communicating designs for "decide what to do with this request": the imperative /chat route (Fig 4.2, live), the rule-based OrchestratorEngine plus its stubbed dispatcher graph (Fig 4.5, orphaned), and the MedicalCoordinatorAgent plus its two-node safety graph (Fig 4.4, orphaned). This migration's purpose is to collapse all three into one LangGraph-native pipeline, with a single LLM node making the routing decision, so that adding a new capability means adding one graph node — not touching a hand-written if/else chain in three different places.

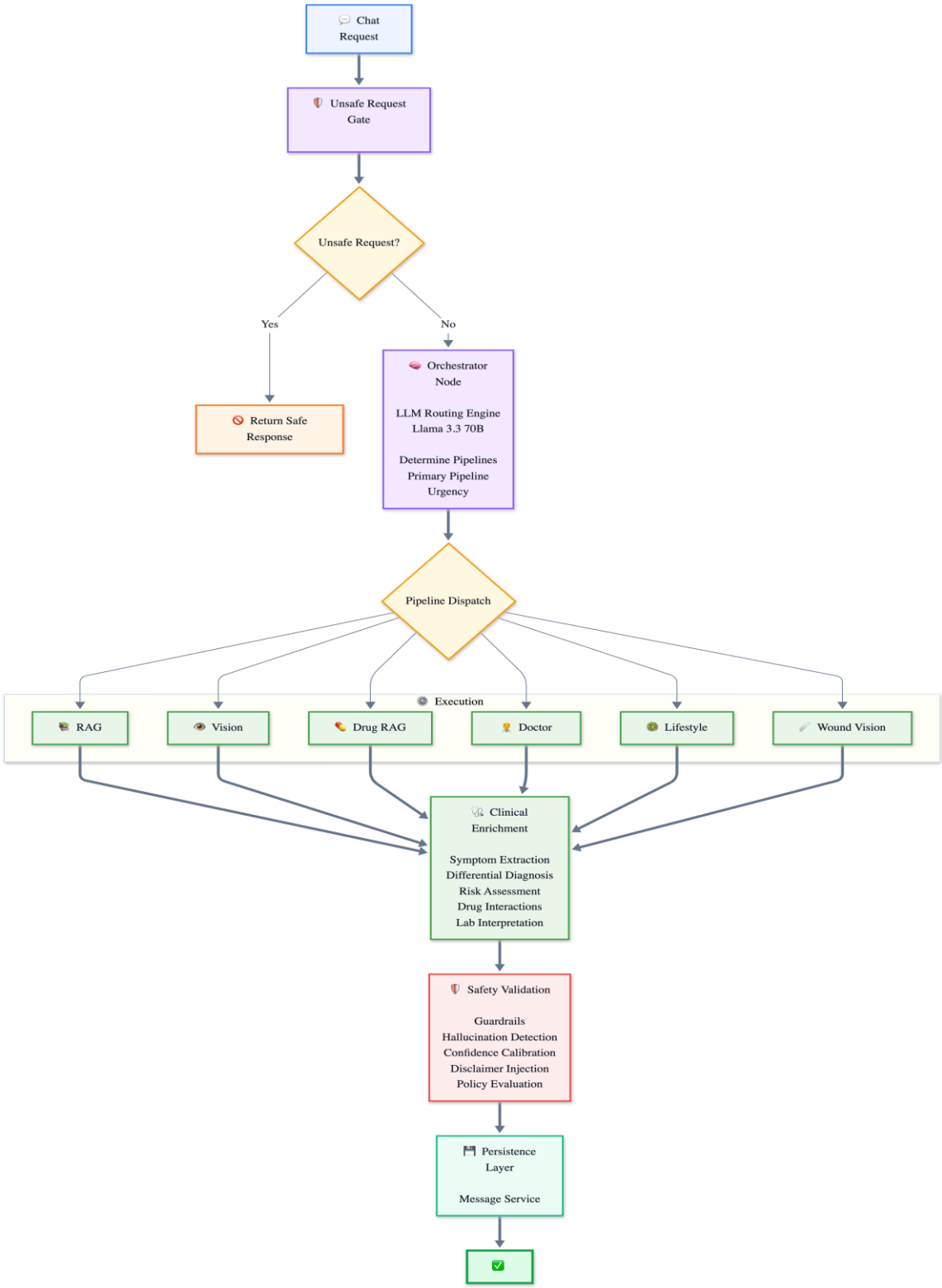
Design principles established

- Router model: llama-3.3-70b-versatile — the same model already used for chat generation (Fig 4.2) and the upload classify+route brain (Fig 4.3), keeping the model surface area small and reusing an already-proven prompting pattern (GroqOrchestratorBrain) for the new orchestration node.
- Seven named pipelines already defined in app/ai/orchestrator/pipelines.py (Pipeline enum): RAG, VISION, DRUG_RAG, DOCTOR_FINDER, LIFESTYLE, WOUND_VISION, STT_PASSTHROUGH — the target graph's conditional edges route to exactly these.
- Reuse working logic as graph nodes rather than rewriting it: ClinicalGenerator + ClinicalRetriever (RAG), VisionService (Vision), get_lifestyle_recommendations (Lifestyle), find_doctors (Doctor Finder) all already work today and just need to be wrapped as LangGraph nodes instead of being called directly from app/api/chat.py.
- Safety becomes a mandatory terminal node. Every branch converges on ResponseValidator before END — this exact converge-on-one-terminal-node pattern is already proven working in the live multimodal graph (Fig 4.3, where vision/ocr/text/lab/drug all converge on finalize_node), so the migration is reusing an established, working pattern rather than inventing a new one.
- UnsafeRequestHandler becomes a real pre-check gate at graph entry. It is fully implemented today (regex-based screening for self-harm, prescribing attempts, definitive-diagnosis requests) but is never invoked before the pipeline runs — the target graph calls it first and short-circuits straight to a safe canned response when it fires.
- PolicyEngine needs real rule evaluation before it can be trusted inside this graph — today it always returns allowed=True regardless of input, which makes it a no-op safety gate.

New node files needed (per current planning)

- Orchestration node — LLM-driven replacement for OrchestratorEngine's keyword-matching (Fig 4.5); open design question below on whether to replace it outright or keep it as a fast pre-filter.
- Drug RAG node — currently Pipeline.DRUG_RAG exists only as an enum value with no retrieval backing; needs an actual retrieval + generation implementation (owner: whoever documents Section 8.1/8.3).
- Wound vision node — a dedicated prompt/flow for dermatological assessment, distinct from the lab-report/prescription-oriented VISION_ANALYSIS_PROMPT (ties directly to the Vision known-gap noted in §7.6; see Section 8.7 for the proposed module).
- Doctor finder node — wraps the existing find_doctors() (NPPES + Google Places) as a graph node; separately, Section 8.4 needs to resolve whether this reconciles with the parallel Pinecone-based Egyptian doctor search (branches/doctor_branch.py).
- Lifestyle node — wraps the existing get_lifestyle_recommendations() as a graph node.
- Clinical enrichment node — promotes ClinicalReasoningAgent's logic (symptom extraction → differential diagnosis → risk assessment → drug interactions → lab interpretation → recommendations, Fig 4.4) from the orphaned agent layer into the live graph.

Target design — Data Flow / Diagram



[Fig 8.6 — Proposed target graph, reconciling Fig 4.2 (live imperative /chat), Fig 4.4 (orphaned agent layer), and Fig 4.5 (orphaned rule-based orchestrator) into one design. Not yet implemented — this is the current working proposal, not a description of shipped code.]

Current Status

Design phase. Three separate implementations exist today (imperative /chat, the agent coordinator, and the rule-based orchestrator with its stubbed dispatcher) and none of them is the final graph. The multimodal upload graph (Fig 4.3) is the closest existing proof that this converge-on-a-terminal-node LangGraph pattern already works reliably in this codebase, and is the template this migration follows.

Known Issues / Next Steps

- Open question: does OrchestratorEngine's keyword-matching get replaced entirely by the new LLM orchestrator node, or kept as a cheap pre-filter that the LLM node only overrides when confidence is low? This is a latency/cost tradeoff that needs a decision before the node is built.
- pipeline_dispatcher_node (Fig 4.5b) needs real implementations behind every placeholder string it currently returns — none of its branches call the real RAG/vision/drug/lifestyle functions yet.
- Reconcile the three state shapes currently in play — ChatState (TypedDict, graph/state.py), OrchestrationState (TypedDict, graph/orchestration_state.py), and AgentContext (Pydantic BaseModel, agents/base.py) — into a single graph state definition.
- Wound-vision node depends on the dedicated wound prompt called out as a gap in §7.6 — sequence this after that prompt exists, not in parallel with it (see Section 8.7).
- Drug-RAG and Doctor-Finder-RAG nodes need actual retrieval backing (see Sections 8.1/8.3/8.4) — the orchestrator node can route to them, but the pipelines themselves don't exist as retrieval-grounded systems yet, only as Pipeline enum members.
- PolicyEngine must get real rule logic (today: always allowed=True) before the safety_node in the target graph can be considered a genuine gate rather than a pass-through.
- Add integration tests asserting that every one of the seven Pipeline enum values is reachable and produces a non-placeholder result once the dispatcher is real.

8.8 Medical Image Analysis Module (Wound / Dermatological Vision)

Purpose

This module allows a user to upload a photo of a visible medical condition — such as a skin rash, wound, or similar external presentation — and receive an AI-generated visual assessment. It is designed to give users a decisive, prioritized impression rather than an unfocused list of possibilities, making the output easier to understand and act on. This is the concrete implementation proposed for the wound_vision pipeline that is currently only a stubbed Pipeline enum member (Section 8.6) and the dedicated dermatological code path flagged as missing in the Vision Pipeline's known gaps (Section 7.6) — today, wound/skin images still flow through the generic VISION_ANALYSIS_PROMPT, which is written for lab reports and prescriptions, not dermatological assessment.

Models Used

The uploaded image is encoded and sent to a vision-capable AI model together with a structured diagnostic prompt. The module is designed with a two-tier resilience strategy so that a temporary outage of one model does not prevent the user from receiving a response — mirroring the same tiered primary/fallback design already used by the general Vision Pipeline (Section 7.1/7.5).

Tier	Component	Role
Primary	Vision-language models (priority-ordered)	Perform the full medical image assessment and generate the structured diagnostic report
Fallback	General-purpose image captioning model	If all primary vision models fail, provides a basic image description with a clear notice that no medical diagnostic capability was used

Data Flow / Diagram

To avoid presenting users with an ambiguous list of equally-weighted possibilities, the module enforces a

fixed, prioritized reporting structure:

- Primary Diagnosis: a single, most-likely condition, stated decisively with a probability indicator and the visual reasoning behind it.
- Alternative Possibilities: two to three additional conditions, ranked from most to least likely, each with a brief visual justification.
- Visual Observations: a factual description of what was observed in the image (color, texture, shape, size, distribution).
- General Care & Treatment Tips: immediate care guidance, safe home/OTC suggestions, things to avoid, and warning signs that warrant urgent care.
- Recommended Next Step: one clear, actionable recommendation (e.g. see a specialist, or seek urgent care under specific conditions).

Every image analysis result concludes with a disclaimer clarifying that it is an AI-generated visual impression, not a medical diagnosis.

Current Status

Designed and documented as a standalone module with a defined prompt structure and two-tier fallback strategy. Not yet wired into the live multimodal graph (Fig 4.3) or the target LangGraph orchestrator (Fig 8.6) as the dedicated wound_vision_node — it is currently proposed content to fill that gap rather than shipped, wired code.

Known Issues / Next Steps

- Wire this module in as the dedicated wound_vision_node called out in the LangGraph migration plan (Section 8.6), distinct from the existing generic VisionProvider/VISION_ANALYSIS_PROMPT (Section 7.4) used for lab reports and prescriptions.
- The prompt design deliberately avoids equal-weight diagnosis lists, which reduces user confusion and the risk of misinterpreting an unranked possibility as the most likely one — this should be validated against real dermatological test images before launch.
- If all primary vision models are unavailable, the system degrades gracefully to a general image description rather than failing outright, and is explicit about the reduced capability of that fallback — confirm this fallback chain reuses the same tenacity retry / timeout pattern already proven in the general Vision Pipeline (Section 7.5) rather than duplicating it.
- Warning signs requiring urgent care are always included when relevant, supporting appropriate escalation to real medical professionals — align the exact wording with the safety/disclaimer layer already live in Section 4.6 so the two don't diverge.

8.9 Foreign Doctor Recommendation System (Google Places API)

Note: Distinct from the Egyptian Doctor Recommendation System (Section 8.5, Pinecone-based). This module targets foreign/international doctor referral using the Google Maps Places API directly.

Purpose

The Doctor Recommendation feature converts a referral suggestion embedded in the model's answer (e.g., "consider seeing a dermatologist") into a concrete, geolocated list of clinics or physicians, using the Google Maps Places API. Architecturally this is the platform's clearest example of grounding an LLM's suggestion in a live external data source rather than the static vector index, since physician directories and clinic listings change far more frequently than a curated medical knowledge base and cannot reasonably be pre-embedded.

Models / Components Used

- extractDoctorReferral.ts (frontend library) — parses the assistant's completion text to detect a physician-specialty referral (e.g., a mentioned specialty, condition-to-specialty mapping, or an explicit "see a ___" recommendation) and extracts a normalized query term.
- useDoctorRecommendations.ts (React hook) — takes the extracted specialty/query and the user's location context, calls the Google Maps Places API (via the Maps JavaScript SDK or a server-side proxy), and returns a list of nearby doctors/clinics with ratings and contact details.
- ResponseCard Doctors tab — renders the returned places as cards (name, specialty, rating, address/contact), giving the user an actionable next step directly beneath the clinical answer.

Data Flow / Diagram

The flow can be summarized in five stages: (1) the chat model produces an answer that may include a referral recommendation as part of its clinical reasoning; (2) extractDoctorReferral.ts scans the completion for referral language and specialty keywords; (3) if a referral is detected, useDoctorRecommendations.ts issues a Places API query scoped to the user's supplied location for that specialty; (4) the Maps API returns a ranked set of places with metadata (name, rating, address, phone, opening hours where available); (5) the frontend renders these results in the Doctors tab, decoupled from — and clearly distinguishable from — the AI-generated clinical text, so the person can tell which part of the response is model-generated reasoning and which part is verified third-party business data.

Interface / Endpoint Specification

Consumer: useDoctorRecommendations.ts (frontend hook)
Upstream trigger: Referral phrase detected by extractDoctorReferral.ts in the chat completion
External API: Google Maps Places API (Places Search / Nearby Search family)
Auth: NEXT_PUBLIC_GOOGLE_MAPS_API_KEY (client-side key, per README env configuration)
Returned fields (typical for this API family): Place name, formatted address, rating, phone number, opening hours, geolocation — exact field mask not confirmed from documentation
Rendering surface: Doctors tab of ResponseCard

Current Status

Separating referral detection (a text-parsing concern) from place lookup (an external-API concern) into two distinct modules is a defensible separation of concerns: it lets the extraction logic be tested independently of network calls, and it lets the Maps integration be swapped or mocked without touching the language-understanding step. Using a real, live directory (Places API) rather than a static, curated list also avoids the staleness problem that would arise from embedding clinic listings into the same Pinecone index used for medical knowledge — clinic hours, addresses, and staffing change on a timescale the RAG pipeline is not built to track.

Known Issues / Next Steps

- No stated internationalization details beyond the Places API's own coverage — location scope depends on what is passed to the query.
- No multi-language support (e.g., Arabic) yet, which limits accessibility of doctor listings for the platform's primary regional audience.
- Log and rate-limit the Places API bridge server-side rather than relying solely on a client-exposed key, reducing key-abuse risk and centralizing location-data handling for privacy auditing.
- Extend the referral-extraction logic with a planned differential-diagnosis scoring so that suggested specialties are derived from a scored clinical signal rather than surface-level keyword/phrase matching.
- Add Arabic-language support to the Places API query/rendering path, consistent with the project's stated multi-language roadmap.
- Privacy note: this flow necessarily shares a location signal (explicit address/city or device geolocation) with a third-party API (Google Maps) — pass only the minimum location granularity needed (e.g., city-level rather than device GPS coordinates) unless the user opts into precise location, and disclose this in the product's privacy notice.

9. Conclusion & Future Work

9.1 Conclusion

Medora demonstrates a working, end-to-end AI-powered healthcare platform that integrates conversational medical assistance, multimodal document understanding, and personalized healthcare recommendations into a single unified system. The project successfully delivers a live, production-functioning /chat pipeline grounded in Retrieval-Augmented Generation, and a fully operational multimodal /upload pipeline that classifies, routes, and processes medical images and documents through a real LangGraph implementation — proving that the core technical approach (vision-first document understanding, RAG-based clinical Q&A, and geo-aware doctor/hospital recommendation) is both feasible and effective for real-world use.
At the same time, the project honestly documents its current architectural debt: several subsystems (the MedicalCoordinatorAgent, the rule-based OrchestratorEngine, and their respective LangGraph wrappers) are fully implemented and independently testable but not yet wired into a live route, the OCR engines are still mocked, and the safety/policy layer is only partially active. This transparency reflects the team's engineering maturity and sets a clear, actionable foundation for the next phase of development.

9.2 Future Work

Based on the architectural audit conducted during this phase, the following priorities are planned for the next development cycle:

- Complete the LangGraph Orchestrator Migration (Section 8.6), consolidating the three separate routing implementations into a single graph-based pipeline with one LLM router node.
- Replace mocked OCR providers (PaddleOCR/EasyOCR) with real inference.
- Activate the safety/policy layer fully — enforce UnsafeRequestHandler as a pre-execution gate and implement real rule evaluation in PolicyEngine.
- Implement the dedicated wound/dermatological vision node (Section 8.9) as a distinct path from generic document vision analysis.
- Build out the Drug-RAG and Doctor-Finder-RAG pipelines with real retrieval backing, replacing their current placeholder status as Pipeline enum members.
- Migrate the database layer from SQLite to PostgreSQL for production scalability.
- Expand testing coverage with quantitative evaluation metrics (RAG retrieval accuracy, response latency, hallucination rate) to validate system performance objectively.
- Conduct real-world usability testing with patients and healthcare professionals to refine the clinical accuracy and tone of AI-generated responses.

10. References

1. National Library of Medicine. *MedlinePlus*. U.S. National Institutes of Health. Retrieved from https://medlineplus.gov/
2. BAAI. *BGE (BAAI General Embedding) Large English v1.5*. Hugging Face Model Hub.
3. Meta AI. *Llama 3.3 70B Instruct*. Retrieved via Groq API.
4. Google. *Gemini 3.5 Flash / 2.5 Flash / 2.5 Pro Documentation*. Google AI for Developers.
5. LangChain / LangGraph Documentation. Retrieved from https://python.langchain.com/ and https://langchain-ai.github.io/langgraph/
6. Pinecone Systems Inc. *Pinecone Vector Database Documentation*. Retrieved from https://docs.pinecone.io/
7. OpenStreetMap Contributors. *Egypt OSM Extract*. Geofabrik. Retrieved from https://download.geofabrik.de/
8. National Plan and Provider Enumeration System (NPPES). U.S. Centers for Medicare & Medicaid Services.
9. PaddlePaddle. *PaddleOCR Documentation*. Baidu Inc.
10. JaidedAI. *EasyOCR Documentation*.
11. FastAPI Documentation. Retrieved from https://fastapi.tiangolo.com/
12. Next.js Documentation. Vercel Inc. Retrieved from https://nextjs.org/docs
13. Apify Technologies s.r.o. *Apify — Web Scraping and Automation Platform*. Retrieved from https://apify.com/

11. Appendix — Model Registry & Known Architectural Debt

11.1 Full model list (from ModelRegistry + Settings)

Task	Model	Provider
Chat / RAG generation	llama-3.3-70b-versatile	Groq
Upload orchestration brain	llama-3.3-70b-versatile	Groq
Reasoning (registered, not actively routed)	qwen-3.6-27b	Groq
Vision (primary)	gemini-3.5-flash	Gemini
Vision (fallback)	gemini-2.5-flash	Gemini
Vision (registered alt.)	gemini-2.5-pro	Gemini
Vision (registered, unused Groq fallback)	llama-4-scout	Groq
Document/structured parsing	gemini-2.5-flash → llama-3.3-70b-versatile fallback	Gemini → Groq
Query rewrite (registered, not actively routed)	llama-3.1-8b	Groq
Drug / Nutrition / Rehab branches	llama-3.3-70b-versatile	Groq
Location extraction (Egyptian doctor search)	llama-3.3-70b-versatile	Groq
Conversation summarization / fact extraction	llama-3.3-70b-versatile	Groq
Speech-to-text	whisper-large-v3-turbo	Groq
Embeddings (medical textbooks)	BAAI/bge-large-en-v1.5	HuggingFace (local)
Embeddings (university regulations, separate project)	intfloat/multilingual-e5-large	HuggingFace (local)
OCR (registered, currently mocked)	paddleocr	local / HuggingFace

11.2 Consolidated known architectural debt

- Multiple retrieval abstractions (embedder, query rewriter, reranker, hybrid search, context formatter, generic retriever) are fully-typed stubs never called by the live RAG pipeline.
- OCR providers return mocked text, not real extraction.
- PolicyEngine always returns allowed=True; UnsafeRequestHandler is fully implemented but never invoked before pipeline execution.
- Several clinical utilities are implemented but disconnected from any live pipeline (see 8.2) — the audit's core finding was that this can happen silently, with no runtime error, which is why explicit wiring verification is required before/after the LangGraph migration.
- graph/builder.py (agent-layer graph) and graph/orchestration_builder.py (rule-based orchestrator graph) are both compiled and independently testable but not invoked by any API route.
- RAG/Pinecone is bypassed entirely whenever a document unified_context is present in /chat.
- Doctor referral reliance on LLM prompt sensitivity alone is unreliable for emotionally-phrased (vs. clinically-phrased) symptom messages — three fixes are proposed (stronger system prompt, hard trigger keywords, backend-side message injection) but not yet implemented.
- The Clinical Reasoning Agent's internal-first/external-fallback retrieval policy (Section 8.1) and the wound/dermatological vision module (Section 8.7) are both newly documented in this revision and still need to be reconciled with, and wired into, the target LangGraph orchestrator (Section 8.6).